

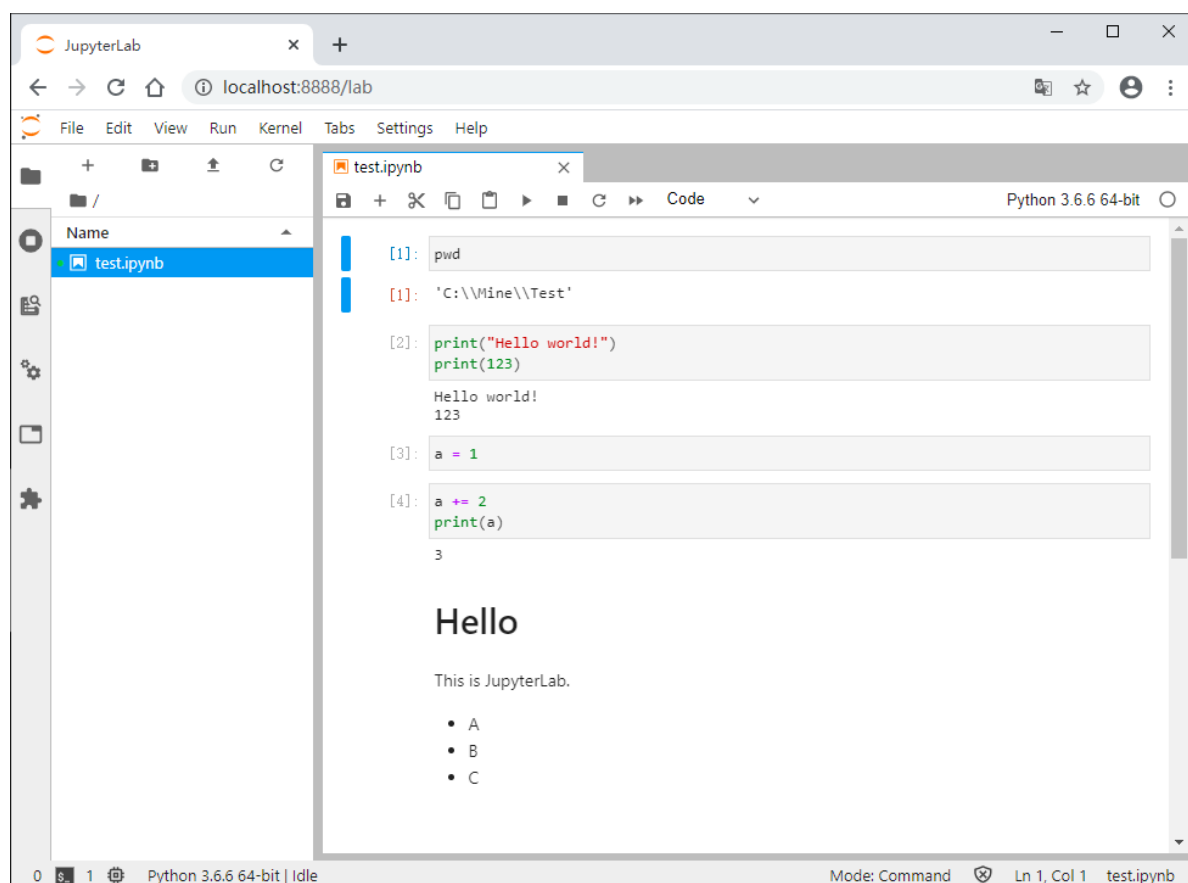
JupyterLab的安装与使用

计算导论 wwd

JupyterLab是什么

如果你知道Jupyter Notebook，则JupyterLab是Notebook的升级版。

JupyterLab（以下可能简称为Jupyter）的使用界面：



从图片可以看到，JupyterLab是Python的一种集成开发环境（IDE），通过它可以解释和运行Python代码。与其他IDE不同的是，JupyterLab使用文本和代码混合的方式编辑文件。

Jupyter系列工具主页：<https://jupyter.org>

准备工作

阅读文章前你需要掌握：

1. 如果你在之前的Python的安装过程中没有将Python路径添加至环境变量，则你需知道将路径添加至环境变量的方法，并将Python路径以及其下的Scripts文件夹添加至环境变量。如果使用的是Anaconda，可忽略这一点。
2. 对使用pip安装库的方法有简单的了解。
3. 知道所在操作系统的终端（命令提示符，PowerShell，Terminal等）如何打开。

验证是否将Python添加至环境变量的方法：

打开cmd，输入 `python`（如果是macOS则输入 `python3`），如果在命令提示符中能够正常打开Python并显示 `>>>` 符号，则说明Python路径已添加至环境变量。若出现“python' 不是内部或外部命令，也不是可运行的程序或批处理文件”等类似错误提示，则说明未添加。

JupyterLab的安装

1. doc: <https://jupyter.org/install>
2. 如果使用的是Anaconda，则请自行查阅上述文档。以下针对pip。
3. 我们使用pip安装JupyterLab。用管理员身份打开cmd（或PowerShell或Terminal，以下不再区分），Windows输入以下命令：

```
1 | python -m pip install jupyterlab
```

或者：

```
1 | py -3 -m pip install jupyterlab
```

如果由于权限原因导致安装失败，则尝试用管理员身份运行cmd再重新输入。

4. 对于macOS，输入

```
1 | pip3 install jupyterlab
```

或者：

```
1 | python3 -m pip install jupyterlab
```

5. 安装完成时，在cmd中输入 `jupyter lab`。如果出现“jupyter' 不是内部或外部命令，也不是可运行的程序或批处理文件”或类似字样，就说明Jupyter所在路径尚未添加至环境变量，无法正常使用。说明你的准备工作没有做完，这时我们需要将Jupyter的路径添加至环境变量。如果能够正常打开，则说明环境变量已配置好。

Jupyter程序所在位置位于Python路径下的Scripts文件夹中，将这一路径添加进环境变量即可。（如果你的Python路径是“C:\Program Files\Python36”，则你的Jupyter路径应为“C:\Program Files\Python36\Scripts”）

6. 环境变量配置完成后，在cmd中输入 `jupyter lab` 开JupyterLab（JupyterLab界面依赖于浏览器）。
7. 点击左上角的“File”按钮下的“shut down”可以退出Jupyter（而不是通过关闭浏览器）。如果依靠关闭浏览器的方式关闭Jupyter的界面，cmd仍然不会退出程序，如需退出Jupyter，需要在cmd中按下 `Ctrl + C` 来终止进程。如果因为不小心而关闭了浏览器，可以从cmd显示的网址信息回到JupyterLab。

JupyterLab的路径

1. 如果未经过特殊配置，JupyterLab的使用目录是当前cmd下的目录。Jupyter的操作范围限于该目录以及其下的子目录和文件。
2. 如果我们需要在其他路径下使用JupyterLab编辑文件，可以在cmd中使用 `cd 路径` 命令跳转至其他的文件夹，再输入 `jupyter lab` 打开JupyterLab。
3. 另一种跳转路径的方法是在命令 `jupyter lab` 后加上路径这一参数：

```
1 | jupyter lab .\Test
```

如果使用这一种方法，当路径中含有空格时，路径前后需加上英文双引号。

4. 在Windows中，为了能够方便地指定某个路径作为工作目录而不是麻烦的cd操作，有两种较为方便的方法（两种方法都先要事先打开相关文件夹）：
 - 使用Shift+右键，会出现“在此处打开PowerShell窗口”选项，利用该选项进入工作目录为当前目录的终端。
 - 在Windows上的路径栏中输入cmd并回车，则可进入工作目录为当前目录的终端。

JupyterLab的使用

Markdown文本编辑

1. Jupyter Notebook 约等于 Python + Markdown
2. Markdown是一种特殊的文本编辑方式。有别于普通的文本编辑，Markdown使用了一些特殊符号（例如 `*`、`#`、`_`、`>`）用来规定文本的格式，十分有利于纯文本文件的排版。如果只使用Markdown书写文本而不对文本格式进行编辑，需要注意一些特殊符号可能会被系统理解成其他功能（一般需要使用转义字符 `\` 来输出这些特殊符号）。
3. Jupyter Notebook中的Markdown支持HTML语法。

IPython

1. IPython的官方简介：An enhanced Interactive Python。
2. IPython基于原本的Python提供了一系列的强化服务，例如支持按下Tab自动补全，在控制台中运行 `ls`，`pwd` 等常用命令。同时IPython是Jupyter系列工具的内核。（Jupyter Notebook曾经名为IPython Notebook。）
3. 你可以在Python路径下的Scripts子文件夹找到IPython（如果你已经安装了IPython或Jupyter）。

```
IPython: C:\Python36\Scripts
Python 3.6.6 (v3.6.6:4cf1f54eb7, Jun 27 2018, 03:37:03) [MSC v.1900 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 7.13.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: pwd
Out[1]: C:\Program Files\Python36\Scripts'

In [2]: ?print
Docstring:
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

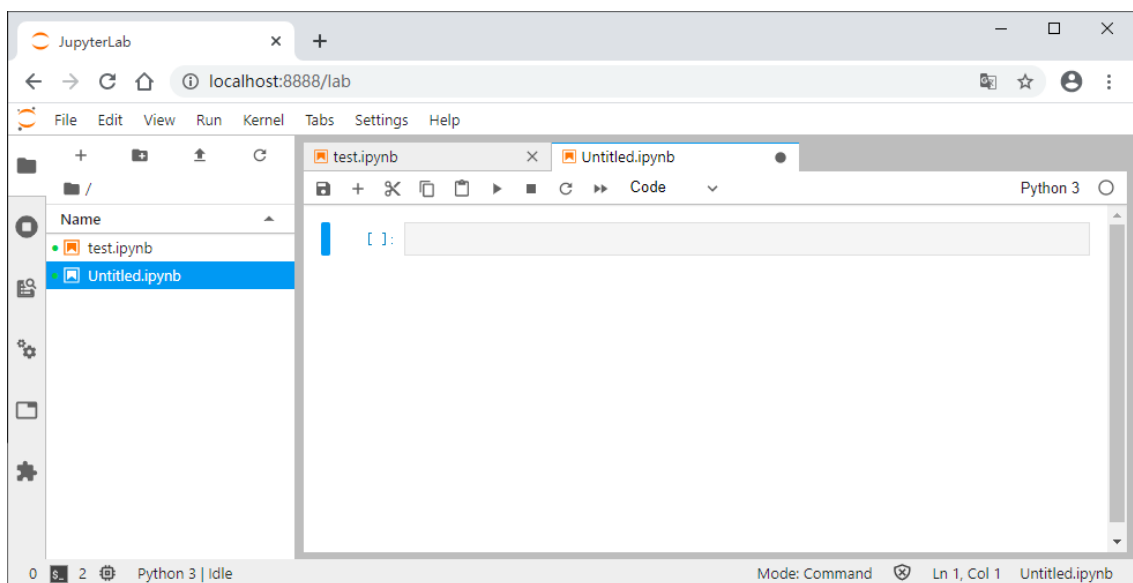
Prints the values to a stream, or to sys.stdout by default.
Optional keyword arguments:
file: a file-like object (stream); defaults to the current sys.stdout.
sep: string inserted between values, default a space.
end: string appended after the last value, default a newline.
flush: whether to forcibly flush the stream.
Type: builtin_function_or_method

In [3]: a = []

In [4]: a.append
append() count() insert() reverse()
clear() extend() pop() sort()
copy() index() remove()
function(object)
```

新建与关闭文件

1. 使用cmd在某一目录（不要使用“C:\WINDOWS\system32”这一目录）下运行jupyter，点击浏览器中的左上角按钮“File”下的“New”，在子菜单中选择“Notebook”，即可新建文件。此时浏览器弹出新标签，进入文本编辑界面。



2. 点击文本编辑界面上方的File菜单，选择“Rename Notebook...”，即可为文件重命名（初始文件名为“Untitled”）。
3. Jupyter Notebook的文件格式为*.ipynb，本质上是一个文本文件。
4. 如果我们需要关闭当前文件，点击文本编辑界面上方的File菜单，选择“Close and Shutdown Notebook”，即可结束并退出当前文件。（如果你的文件没有在运行，也可以选择直接点击关闭图标）

编辑文件

1. Jupyter Notebook采用了文本分区的形式，不同的区块可以选择“Code”、“Markdown”模式，分别代表Python代码、Markdown文本。
2. 使用控件栏上的  号新增一个区块。新增后可在右侧的下拉框中选择使用Code还是Markdown模式。使用Code模式时，区块左侧会出现“[]”标签。
下拉框内还有另外一种模式：“Raw”。而“Raw”模式下编辑的文本不会被运行或解释为Markdown，因此可以替代“Markdown”进行无格式文本的编辑（如果用户不熟悉Markdown的形式，可以采用此模式输入文本）。

运行文件

1. 如果选中Code区块，点击控件栏上的“运行”按钮，即可运行区块内的Python代码。可看到文本输出在代码下方。
2. 如果运行Markdown模式下的区块，可以看到文本根据Markdown的标签被转化为相应形式。
3. Raw区块无法运行。
4. 不同Code区块可有联系（只要保证不同代码区块间的运行次序即可）。代码运行的先后顺序以每个区块前的数字进行表示。（即可以先运行后面的代码，再运行前边的代码，虽然一般不会这么做。）
5. 如果我们需要删除某一区块，可选中该区块，右键选择“Delete Cells”选项。或者使用快捷键。
6. 选择控件栏或菜单栏File的相关按钮对文件进行保存。也可使用相应按钮对区块进行剪切复制粘贴等操作。

快捷键

使用菜单栏进行一些命令的使用可能十分烦琐，可以使用快捷键来替代这些操作。

使用非组合型快捷键的前提是选中一个区块但不让其进入输入状态（即不出现文本编辑光标）。可以通过鼠标点击区块左侧实现。如果正处于该区块的输入状态，按下键盘上的 **Esc** 键退出输入状态。

退出输入状态后，可通过键盘上的快捷键对区块进行操作。快捷键可查看菜单栏Help下的“Keyboard Shortcuts”。下方列出部分快捷键：

- **Enter** : 转入编辑模式
- **Shift-Enter** : 运行本单元，选中下个单元
- **Ctrl-Enter** : 运行本单元
- **Alt-Enter** : 运行本单元，在其下插入新单元
- **Y** : 单元转入Code状态
- **M** : 单元转入Markdown状态
- **R** : 单元转入Raw状态
- **Up / K** : 选中上方单元
- **Down / J** : 选中下方单元
- **A** : 在上方插入新单元
- **B** : 在下方插入新单元
- **X** : 剪切选中的单元
- **C** : 复制选中的单元
- **Shift-V** : 粘贴到上方单元
- **V** : 粘贴到下方单元
- **Z** : 恢复删除的最后一个单元
- **D,D** : 删除选中的单元

- **S**:保存文件
- **L**: 显示或隐藏行号